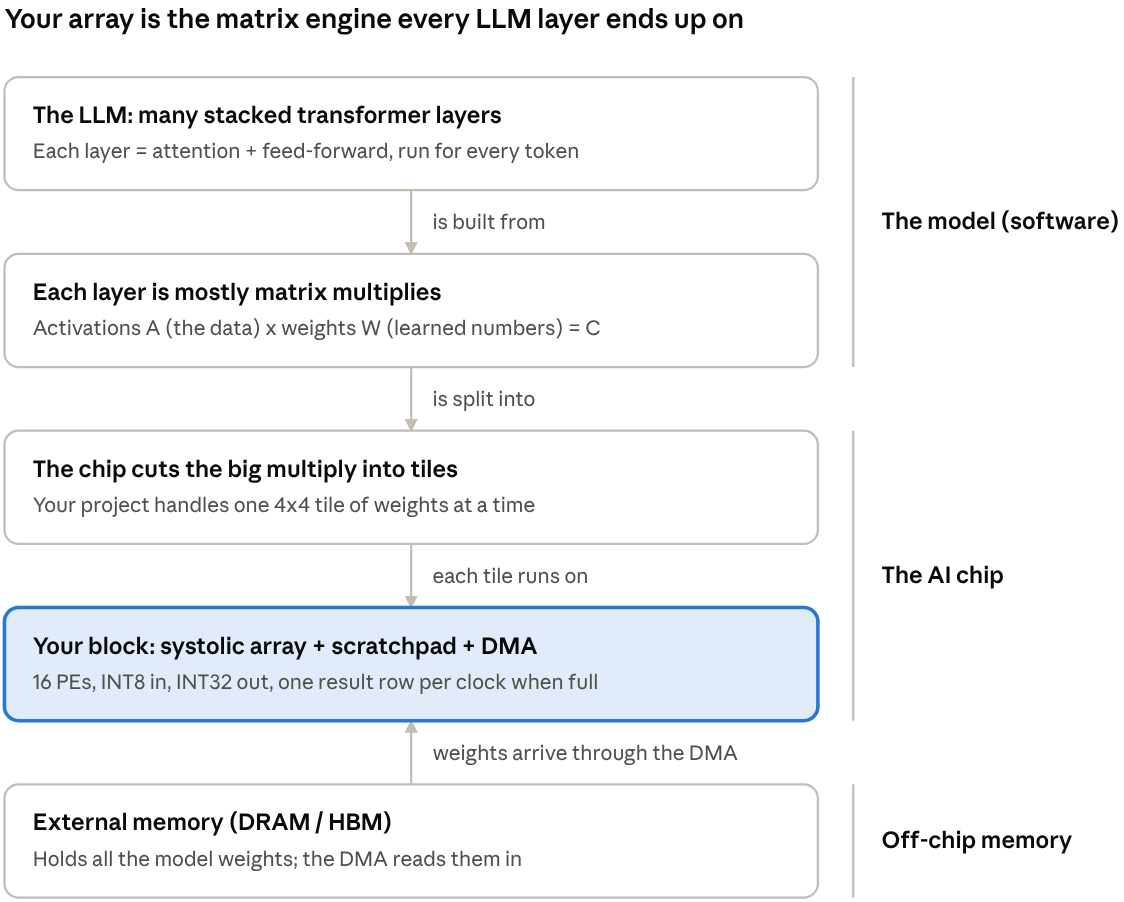
Big Picture: What the Systolic Array Project Is For

Nov 11, 2026 · @Kiran Nagendra

The one-paragraph picture

You are building a small version of the **matrix-multiply engine**, the block of an AI chip that does almost all the arithmetic when an LLM runs. An LLM is mostly one operation repeated billions of times: multiply a big grid of learned numbers (weights) by a grid of the current data (activations). Your project is the hardware that does that multiply quickly, plus the memory plumbing that feeds it, plus Python models that prove it is correct and predict its speed. A real chip such as Google's first TPU uses the same idea with a 256 x 256 grid of PEs, and your 4x4 version is the same design at toy size.

Where it sits in an LLM



LLM to PE array stack · 5 layers

The LLM is software, but every layer of it turns into matrix multiplies that the chip has to run. The highlighted box is what you are building. Everything above it is the reason it exists, and the memory below it is what limits how fast it can go.

What you are designing

You design four pieces. Each is small, and together they form one accelerator block.

Piece	What it is	Why it exists
PE (processing element)	One multiply-accumulate unit that holds one weight: psum_out = psum_in + a_in x weight	The smallest unit of arithmetic. Everything else is built from copies of it

Piece	What it is	Why it exists
Systolic array (4x4)	16 PEs wired in a grid. Activations flow east, partial sums flow south	Reuses each number many times as it passes through, so memory is read once instead of 16 times
Scratchpad and DMA	A small on-chip memory plus an engine that copies data in from outside while the array computes	The array is only useful if it never waits for data. This is where memory-controller experience applies
Python golden model and performance model	An answer key, plus formulas that predict how busy the array stays	The golden model proves the RTL is correct. The performance model shows you understand what limits speed

Another way to say it: the PE is the arithmetic, the array is the arithmetic arranged for reuse, the scratchpad and DMA keep it fed, and the Python models check and explain it.

Exact inputs and outputs

The block computes one matrix product, C = A x W, and the three matrices have fixed roles.

Matrix	Meaning in an LLM	Shape for your 4x4 array	Number type	How it enters
W	A learned weight matrix from the model	4 x 4 (K x N)	INT8	Loaded once through the write port, then held inside the PEs
A	Activations: the current tokens' data	M x 4 (M rows)	INT8	Streamed in from the left, one row per clock, skewed
C	The layer's output	M x 4	INT32	Comes out of the bottom of each column

Outputs are 32 bits because each product is 16 bits and the sum of many products would overflow 16 bits.

A tiny example with a 2x2 version, so you can check it by hand: W = [[1, 2], [3, 4]], and two activation rows A0 = [5, 6] and A1 = [7, 8].

- C[0][0] = 5 x 1 + 6 x 3 = 23
- C[0][1] = 5 x 2 + 6 x 4 = 34
- C[1][0] = 7 x 1 + 8 x 3 = 31
- C[1][1] = 7 x 2 + 8 x 4 = 46

Your array produces exactly these numbers. Each PE does one of the multiplications, and the partial sums added down a column produce each output. In your testbench, `gemm_int8` in the golden model computes this same product, and the RTL has to match it bit for bit.

How it runs in real time

Here is what happens, in order, when the model needs one matrix multiply for one layer.

1. **A controller decides the job.** Software on the chip says: multiply these activations by this weight matrix.
2. **DMA copies weights in.** The DMA engine reads a 4x4 block of weights from outside memory into the on-chip scratchpad.
3. **Weights are loaded into the PEs.** The write port sets each PE's stored weight. This takes 16 clocks with a single write port, and it is paid once per weight block.
4. **DMA streams activations in.** While the array works, the DMA keeps copying the next rows of activations into the scratchpad.
5. **Skew logic feeds the array.** Row r of the activations is delayed r clocks so every activation meets its partial sum.
6. **The array computes.** Each clock, every PE with data does one multiply-accumulate. A wave of work moves from the top-left to the bottom-right, as in the clock-by-clock animation.
7. **Answers come out of the bottom.** After de-skewing, one output row appears per clock once the pipeline is full.
8. **Outputs are kept or passed on.** They are added to earlier partial results if the matrix was bigger than the array, and then go to the next stage of the layer.

The key point is that steps 4 to 7 overlap. The array computes on one batch while the DMA fetches the next, which is called double buffering.

From your 4x4 to a real chip

Real weight matrices are far bigger than 4x4, so the array works on one tile at a time. A 4096 x 4096 weight matrix, a typical LLM layer size, splits into (4096 / 4) x (4096 / 4) = 1,048,576 tiles on your array, or 16 x 16 = 256 tiles on a 256 x 256 array. For each tile the chip loads the weights, streams activations through, and adds the new partial sums to the earlier ones.

	Your project	Real accelerator (TPU v1 style)
Array size	4 x 4 = 16 PEs	256 x 256 = 65,536 PEs
Peak MACs per clock	16	65,536
Number format	INT8 in, INT32 out	INT8 in, 32-bit accumulation
Memory	Small scratchpad and DMA	Large on-chip buffers plus external memory

The hard part at real scale is feeding the array. Arithmetic is cheap, and moving numbers from memory costs far more time and energy. When a model generates one token at a time, each weight is used only once, so the chip spends most of its time waiting for memory instead of multiplying. That is why accelerator interviews focus on memory bandwidth, tiling and reuse, and why the performance model in your project is as important as the RTL.

Explaining it in 30 seconds

A version you can say out loud: "LLMs spend nearly all their compute on matrix multiplies, so AI chips are built around a grid of multiply-accumulate units called a systolic array. I built a small weight-stationary INT8 version in SystemVerilog, with a scratchpad and DMA to keep it fed, a Python golden model to verify it bit for bit, and a performance model that predicts its utilization. I compared the model's cycle counts with the RTL and explained the differences."

Questions you should be ready for:

- Why weight-stationary, and what would change with output-stationary?
- Why is the accumulator 32 bits?
- Where do the fill and drain clocks come from?
- What limits the speed when the array is scaled up?